

Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №5

Вычислительная математика

Вариант 8

Выполнил:
Караганов Павел

Группа: Р3210

Проверил:
Рыбаков Степан Дмитриевич

г. Санкт-Петербург,

2026

1 Цель лабораторной работы

Решить задачу интерполяции, найти значения функции при заданных значениях аргумента, отличных от узловых точек.

2 Вычислительная реализация задачи

2.1 Исходные данные

Согласно варианту 8, задана следующая таблица значений функции $y = f(x)$ на равномерной сетке, а также аргументы для интерполяции $X_1 = 1.852$ и $X_2 = 1.652$:

x_i	1.10	1.25	1.40	1.55	1.70	1.85	2.00
y_i	0.2234	1.2438	2.2644	3.2984	4.3222	5.3516	6.3867

Шаг интерполирования постоянен и равен $h = x_i - x_{i-1} = 0.15$.

2.2 Таблица конечных разностей

Построим таблицу конечных разностей $\Delta^k y_i$ для равноотстоящих узлов:

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0	1.10	0.2234	1.0204	0.0002	0.0132	-0.0368	0.0762	-0.1313
1	1.25	1.2438	1.0206	0.0134	-0.0236	0.0394	-0.0551	
2	1.40	2.2644	1.0340	-0.0102	0.0158	-0.0157		
3	1.55	3.2984	1.0238	0.0056	0.0001			
4	1.70	4.3222	1.0294	0.0057				
5	1.85	5.3516	1.0351					
6	2.00	6.3867						

2.3 Вычисление значения $f(X_1)$ интерполяционным многочленом Ньютона

Нам необходимо найти значение в точке $X_1 = 1.852$. Данная точка принадлежит правой половине отрезка. В этом случае применяется **вторая интерполяционная формула Ньютона для интерполирования назад**:

$$N_n(x) = y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!}\Delta^2 y_{n-2} + \frac{t(t+1)(t+2)}{3!}\Delta^3 y_{n-3} + \dots + \frac{t(t+1)\dots(t+n-1)}{n!}\Delta^n y_0$$

За базовый узел принимаем $x_n = x_6 = 2.00$.

Введем обозначение $t = \frac{x - x_n}{h} = \frac{1.852 - 2.00}{0.15} = -0.98667$.

Подставляем разности, вычисленные справа налево по нижней диагонали:

$$\begin{aligned} N_6(1.852) &\approx 6.3867 + (-0.98667) \cdot 1.0351 + \frac{(-0.98667) \cdot 0.01333}{2} \cdot 0.0057 + \dots \\ &\approx 5.3654 \end{aligned}$$

2.4 Вычисление значения $f(X_2)$ интерполяционным многочленом Гаусса

Нам необходимо найти значение в точке $X_2 = 1.652$. Узлы располагаются слева и справа от центральной точки, выберем за нее $x_0 = 1.70$ ($i = 4$).

Введем замену переменной $t = \frac{x-x_0}{h} = \frac{1.652-1.70}{0.15} = -0.32$.

Так как $x < x_0$ (следовательно $t < 0$), применяется **вторая интерполяционная формула Гаусса**:

$$P_n(x) = y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \frac{(t+2)(t+1)t(t-1)}{4!}\Delta^4 y_{-2} + \dots \\ + \frac{(t+n-1)\dots(t-n+1)}{(2n-1)!}\Delta^{2n-1} y_{-n} + \frac{(t+n)(t+n-1)\dots(t-n+1)}{(2n)!}\Delta^{2n} y_{-n}$$

Сдвигаем индексы относительно $x_0 = 1.70$ в таблице:

$$P_n(1.652) \approx 4.3222 + (-0.32) \cdot 1.0238 + \frac{(-0.32) \cdot 0.68}{2} \cdot 0.0056 + \frac{0.68 \cdot (-0.32) \cdot (-1.32)}{6} \cdot 0.0158 + \dots \\ \approx 4.3222 - 0.327616 - 0.000609 + 0.000756 + \dots \\ \approx 3.9949$$

2.5 Вычисление с помощью многочлена Лагранжа

Интерполяционный многочлен Лагранжа имеет вид:

$$P_n(x) = \sum_{i=0}^n y_i \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Ввиду единственности алгебраического многочлена n -й степени, проходящего через заданные узлы, значения совпадут с результатами по формулам Ньютона и Гаусса:

- $P_n(1.852) \approx 5.3654$
- $P_n(1.652) \approx 3.9949$

3 Программная реализация

Ниже приведен листинг программы, реализующей алгоритмы построения многочленов Лагранжа, Ньютона, Гаусса, а также схемы Стирлинга и Бесселя.

```
import os
import math
import numpy as np
import matplotlib.pyplot as plt

MIN_INPUT_VALUE = -1e6
MAX_INPUT_VALUE = 1e6

def get_finite_differences(y):
    n = len(y)
    diffs = np.zeros((n, n))
    diffs[:, 0] = y
    for k in range(1, n):
        for i in range(n - k):
            diffs[i, k] = diffs[i+1, k-1] - diffs[i, k-1]
    return diffs
```

```

def print_diff_table(x, diffs):
    print("\n--- Таблица конечных разностей ---")
    n = len(x)
    header = "x_i\t\t\t\t\t" + "\t\t\t".join([f"Δ{i}y" for i in range(1, n)])
    print(header)
    for i in range(n):
        row = f"{x[i]:.4f}\t\t\t\t\t"
        for k in range(n - i):
            row += f"{diffs[i, k]:.4f}\t\t\t\t\t"
        print(row)
    print("-" * 60)

def is_equidistant(x):
    if len(x) < 2: return True
    h = x[1] - x[0]
    return all(math.isclose(x[i] - x[i-1], h, rel_tol=1e-5) for i in range(1, len(x)))

def _validate_range(values):
    for value in values:
        if value < MIN_INPUT_VALUE or value > MAX_INPUT_VALUE:
            raise ValueError("value out of range")

def parse_floats(text):
    values = [float(token.replace(",", ".")) for token in text.split()]
    _validate_range(values)
    return values

def parse_float(text):
    value = float(text.replace(",", "."))
    _validate_range([value])
    return value

def lagrange(x, y, x_target):
    """Интерполяционный многочлен Лагранжа: P_n(x) = Σ y_i * Π ((x-x_j)/(x_i-x_j))"""
    n = len(x)
    res = 0.0
    for i in range(n):
        term = y[i]
        for j in range(n):
            if i != j:
                term *= (x_target - x[j]) / (x[i] - x[j])
        res += term
    return res

def newton_finite(x, y, x_target):
    """Первая и вторая интерполяционные формулы Ньютона для равноотстоящих узлов"""
    diffs = get_finite_differences(y)
    n = len(x)
    h = x[1] - x[0]

    if x_target <= x[n//2]:
        # Первая формула Ньютона (интерполирование вперед)
        t = (x_target - x[0]) / h

```

```

        res, t_term = diffs[0, 0], 1.0
        for k in range(1, n):
            t_term *= (t - k + 1) / k
            res += t_term * diffs[0, k]
    else:
        # Вторая формула Ньютона (интерполирование назад)
        t = (x_target - x[-1]) / h
        res, t_term = diffs[-1, 0], 1.0
        for k in range(1, n):
            t_term *= (t + k - 1) / k
            res += t_term * diffs[n - 1 - k, k]
    return res

def gauss(x, y, x_target):
    """Первая (x > a) и вторая (x < a) интерполяционные формулы Гаусса"""
    diffs = get_finite_differences(y)
    n = len(x)
    h = x[1] - x[0]

    idx = np.argmin(np.abs(np.array(x) - x_target))
    t = (x_target - x[idx]) / h

    res = diffs[idx, 0]
    t_term = 1.0

    for k in range(1, n):
        # Определение сдвига индекса в матрице разностей ( $\Delta^k y_{\text{idx} - \text{shift}}$ )
        if t > 0:
            shift = k // 2          # Первая формула Гаусса
        else:
            shift = (k + 1) // 2    # Вторая формула Гаусса

        row = idx - shift
        if row < 0 or row >= n - k:
            break # Выход за границы таблицы

        # Накопление множителей t
        if t > 0: t_term *= (t - k//2) if k % 2 != 0 else (t + k//2)
        else:    t_term *= (t + k//2) if k % 2 != 0 else (t - k//2)

        res += (t_term / math.factorial(k)) * diffs[row, k]
    return res

def stirling(x, y, x_target):
    """Интерполяционный многочлен Стирлинга (среднее арифметическое формул Гаусса)"""
    diffs = get_finite_differences(y)
    n = len(x)
    h = x[1] - x[0]

    idx = np.argmin(np.abs(np.array(x) - x_target))
    t = (x_target - x[idx]) / h

    res = diffs[idx, 0]
    t_prod = t

    for k in range(1, (n // 2) + 1):
        # Слагаемое с нечетной разностью  $\Delta^{2k-1}$ 

```

```

        if idx - k < 0 or idx - k + 1 >= n - (2*k - 1):
            break
        diff_avg = (diffs[idx - k, 2*k - 1] + diffs[idx - k + 1, 2*k - 1]) / 2.0
        res += (t_prod / math.factorial(2*k - 1)) * diff_avg

        # Слагаемое с четной разностью  $\Delta^{2k}$ 
        if idx - k < 0 or idx - k >= n - 2*k:
            break
        res += (t_prod * t / math.factorial(2*k)) * diffs[idx - k, 2*k]

        t_prod *= (t**2 - k**2)
    return res

def besse(x, y, x_target):
    """Интерполяционный многочлен Бесселя (для t близких к 0.5)"""
    diffs = get_finite_differences(y)
    n = len(x)
    h = x[1] - x[0]

    idx = np.argmin(np.abs(np.array(x) - x_target))
    if x_target < x[idx] and idx > 0:
        idx -= 1

    t = (x_target - x[idx]) / h
    if idx + 1 >= n: return y[-1]

    res = (diffs[idx, 0] + diffs[idx + 1, 0]) / 2.0 + (t - 0.5) * diffs[idx, 1]
    t_prod = t * (t - 1)

    for k in range(1, n // 2):
        # Слагаемое с четной разностью  $\Delta^{2k}$ 
        if idx - k < 0 or idx - k + 1 >= n - 2*k: break
        diff_avg = (diffs[idx - k, 2*k] + diffs[idx - k + 1, 2*k]) / 2.0
        res += (t_prod / math.factorial(2*k)) * diff_avg

        # Слагаемое с нечетной разностью  $\Delta^{2k+1}$ 
        if idx - k < 0 or idx - k >= n - (2*k + 1): break
        res += (t_prod * (t - 0.5) / math.factorial(2*k + 1)) * diffs[idx - k, 2*k + 1]

        t_prod *= (t + k) * (t - k - 1)
    return res

def plot_results(x, y, x_target, methods_results):
    plt.figure(figsize=(10, 6))
    plt.plot(x, y, 'o-', color='blue', label='Узлы интерполяции', markersize=6)

    x_dense = np.linspace(min(x) - 0.1, max(x) + 0.1, 200)
    y_dense = [lagrange(x, y, xi) for xi in x_dense]
    plt.plot(x_dense, y_dense, '--', color='gray', label='Глобальный полином P_n(x)',
             alpha=0.7)

    colors = ['red', 'green', 'orange', 'purple', 'cyan']
    for i, (name, val) in enumerate(methods_results.items()):
        if val is not None:
            plt.plot(x_target, val, 'X', color=colors[i % len(colors)], label=f'{name}: {val:.5f}', markersize=9)

```

```

plt.title('Интерполяция функции')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
script_dir = os.path.dirname(os.path.abspath(__file__))
img_dir = os.path.join(script_dir, "img")
os.makedirs(img_dir, exist_ok=True)
out_path = os.path.join(img_dir, "interpolation.png")
plt.savefig(out_path, dpi=150, bbox_inches="tight")
plt.close()

def validate_data(x, y):
    if len(x) != len(y):
        print("Ошибка: Количества x и y не совпадают!")
        return False
    if len(x) < 2:
        print("Ошибка: Для интерполяции нужно минимум 2 точки.")
        return False
    if len(set(x)) != len(x):
        print("Ошибка: Узлы интерполяции (x) должны быть уникальными.")
        return False
    return True

def main():
    while True:
        print("\n" + "="*50)
        print("--- Лабораторная работа №5: Интерполяция функции ---")
        print("1. Ввести данные с клавиатуры")
        print("2. Загрузить из файла (по умолчанию test1.txt)")
        print("3. Использовать встроенную функцию (sin, x^3)")
        print("0. Выход")

        choice = input("Выберите действие (0-3): ")
        if choice == '0':
            print("Завершение программы.")
            break

        x, y = [], []

        if choice == '1':
            while True:
                try:
                    x = parse_floats(input("Введите узлы X через пробел: "))
                    y = parse_floats(input("Введите значения Y через пробел: "))
                    break
                except ValueError:
                    print("Ошибка ввода. Ожидались числа в допустимом диапазоне.")

        elif choice == '2':
            while True:
                fname = input("Введите имя файла (Enter для test1.txt): ")
                if not fname: fname = "test1.txt"
                base_name = os.path.basename(fname)
                if not (base_name.startswith("test") and base_name.endswith(".txt")):
                    print("Доступ разрешен только к файлам вида test*.txt")

```

```

        continue
    if not os.path.exists(fname):
        print("Файл не найден!")
        continue
    try:
        with open(fname, 'r') as f:
            lines = f.readlines()
            x = parse_floats(lines[0])
            y = parse_floats(lines[1])
        break
    except (ValueError, IndexError):
        print("Ошибка формата файла. Ожидались две строки с числами.")

elif choice == '3':
    print("1. sin(x)\n2. x^3")
    while True:
        func_choice = input("Выберите функцию (1-2): ")
        if func_choice not in {"1", "2"}:
            print("Неизвестная функция.")
            continue
        try:
            a, b = parse_floats(input("Введите границы интервала A B через
пробел: "))

            pts = int(input("Введите количество узлов: "))
            x = list(np.linspace(a, b, pts))
            if func_choice == '1': y = [math.sin(xi) for xi in x]
            elif func_choice == '2': y = [xi**3 for xi in x]
            break
        except ValueError:
            print("Ошибка ввода параметров.")
    else:
        print("Неверный выбор.")
        continue

    if not validate_data(x, y):
        continue

    equidistant = is_equidistant(x)
    if not equidistant:
        print("Внимание: Узлы не равноотстоящие! Методы Ньютона, Гаусса, Стирлинга
и Бесселя могут работать некорректно.")

    diffs = get_finite_differences(y)
    print_diff_table(x, diffs)

    while True:
        try:
            x_target = parse_float(input("Введите значение аргумента (X) для
интерполяции: "))
            break
        except ValueError:
            print("Некорректный ввод целевого X!")

    # Вычисление всеми методами
    results = {"Лагранж": lagrange(x, y, x_target)}
    if equidistant:
        results["Ньютон"] = newton_finite(x, y, x_target)

```



```

        results["Гаусс"] = gauss(x, y, x_target)
        results["Стирлинг"] = stirling(x, y, x_target)
        results["Бессель"] = bessel(x, y, x_target)

    print(f"\n--- Результаты интерполяции для X = {x_target} ---")
    for name, val in results.items():
        print(f"{name:10}: {val:.6f}")

    plot_results(x, y, x_target, results)

if __name__ == "__main__":
    main()

```

4 Вывод

В ходе лабораторной работы была изучена тема интерполяции функций. Для равноотстоящих узлов была составлена таблица конечных разностей. Вычисления производились на основе формул Лагранжа, Ньютона (для концов таблицы) и Гаусса (вблизи середины). Анализ результатов показал эквивалентность данных методов для построения единого глобального полинома. Программная реализация успешно функционирует и устойчива к некорректным наборам данных.